is largely reactive, identifying issues after they occur, whereas observability enables a more proactive approach by helping organisations build reliable and resilient systems.

This shift has become increasingly important with the adoption of microservices architectures. In traditional monolithic architecture, all application modules are maintained within a single codebase. Even a small change in one module requires rebuilding and deploying the entire application, making updates slower and more complex. Microservices address this gap by decoupling application components into independent services that can be developed, managed and deployed separately. For example, an e-commerce platform such as Amazon involves multiple services, including product search, cart management, payment processing and email notifications. In a microservices environment, each of these functions operates independently, allowing a change in one service, such as the cart module, to be updated without impacting the others.

However, distributed microservices environments also introduce greater complexity, making observability essential. Organisations need to capture and analyse the three key components: logs, traces and metrics. Along with this, chaos engineering has emerged as an important practice to test system resilience.

Chaos engineering: Building resilient Kubernetes systems

Chaos engineering helps organisations understand how systems behave during unexpected failures by intentionally introducing controlled disruptions and analysing the response. Popularised by Netflix through its backend reliability experiments, the approach focuses on improving system resilience and maintaining consistent performance.

The process involves defining the system's steady state and then introducing controlled failures, such as shutting down nodes, deleting pods, increasing traffic, adding latency or creating resource constraints. For example, if an application runs across 10 nodes, teams may intentionally take down two nodes to observe how the system responds. By comparing performance before and after the failure, organisations can identify weaknesses and improve reliability. Tools such as Netflix's Monkey are used to simulate these failure scenarios.

Although chaos engineering requires additional resources, infrastructure and skilled teams, it has become increasingly important with the adoption of microservices and distributed architectures. Organisations use it to prepare for unpredictable traffic spikes and large-scale events such as Big Billion Days, Great Indian Festival sales or high-demand streaming events. Platforms such as Hotstar have benefited from similar reliability practices to deliver smoother experiences during major live events.

However, the engineering must be performed carefully, as failures are injected into running systems. Experiments need to be controlled and planned rather than randomly disrupting services. For example, shutting down most of the nodes in a cluster would not provide meaningful insights. Instead, teams continuously monitor system behaviour, collect data and use the findings to improve resilience.

Concept of "Monkey" refers to a set of tools or services that simulate various failures and disruptions in a system to test its resilience and stability

Chaos engineering

The Modern Kubernetes Reality & Challenges

Scalability & Performance: Difficulty in managing dynamic workloads

Debugging Issues: Containers are ephemeral, making troubleshooting harder

Security & Compliance: Visibility into workloads and network policies

Resource Optimization: Identifying inefficiencies in CPU/memory usage

Service Mesh complexity : Understanding and managing service to service communication

Image: A visual representation of a complex microservices architecture

The challenges in Kubernetes environments